Nirma University
Institute of Technology



Course Code :- 6CS103CC26
Course Name :- Operating System

Innovative Assignment - 1

Student Name
Vinay Thakor

Student Roll No.
26MCA061

Faculty In Charge
Prof. Drashti Patel

SMART CPU SCHEDULER

A CPU Scheduling Simulator with Workload-Based Algorithm Recommendation

Operating Systems Innovative Assignment 1 Report

---

1. Introduction

CPU scheduling decides which ready process receives the processor and for how long; in turn, Tackling Gantt-chart questions by hand shows how one algorithm behaves on one example, broadly speaking; though, it doesn't show how a few algorithms compare on the same workload; this project is employing the scheduling concepts to a applied obstacle by building a simulator in Java, at least empirically.

The program accepts the Process ID, Arrival Time, Burst Time and Priority of each process and schedules them under the seven policies - FCFS, SJF, SRTF, Priority (non-preemptive), Priority (preemptive), Priority with Aging, and Round Robin, all things considered; in fact, what it creates is the Gantt chart automatically and finds Average Waiting Time, Average Turnaround Time, Average Response Time, CPU Utilization, Context Switches and Output, at least for now. The extra trait is a workload-based algorithm recommender that scores all algorithms and indicates the most fitting one, supported by priority aging, random workload generation and a graphical visualization; three workloads (short jobs, mixed jobs and high-priority jobs) were employed for testing.

2. Program Design and Data Structures

The whole application is usually written in one Java file, SmartCPUScheduler.java. The scheduling logic is completely separate from the interface: the Engine class calculates, the Report class formats the results. What's more, the interface only shows them. With that separation the simulator can also run without a window; starting the program with the --report option prints the full test report to the console. The primary building blocks are:

- **Algo (enum):** is holding the seven algorithms, each with a title, a type tag (to make "Preemptive, time quantum") and a one-line description, in real-world scenarios.
- **Proc:** one process with its input fields (id, arrival time at, burst time bt, priority pri) and its working fields, namely remaining time rem, completion time ct, response time rt, effective priority eff and waiting counter streak, on the whole; the copy() method gives every algorithm a fresh copy, so all seven algorithms are run on same input, in the main.
- **Slice:** one continuous block of the Gantt chart, storing the process id (or idle), start time and end time, in actual application.
- **Result is storing** the list of slices, the rows, all averages, CPU utilization, output, context switches, longest waiting time, makespan and the recommendation score, at least in principle; in fact, Row and Result, including Row stores the per-process completion, turnaround, waiting and response times.
- **Preset workloads, including** the three test workloads are stored in the PRESETS array as arrival and burst triples and are loaded by preset(i), at least in practice.

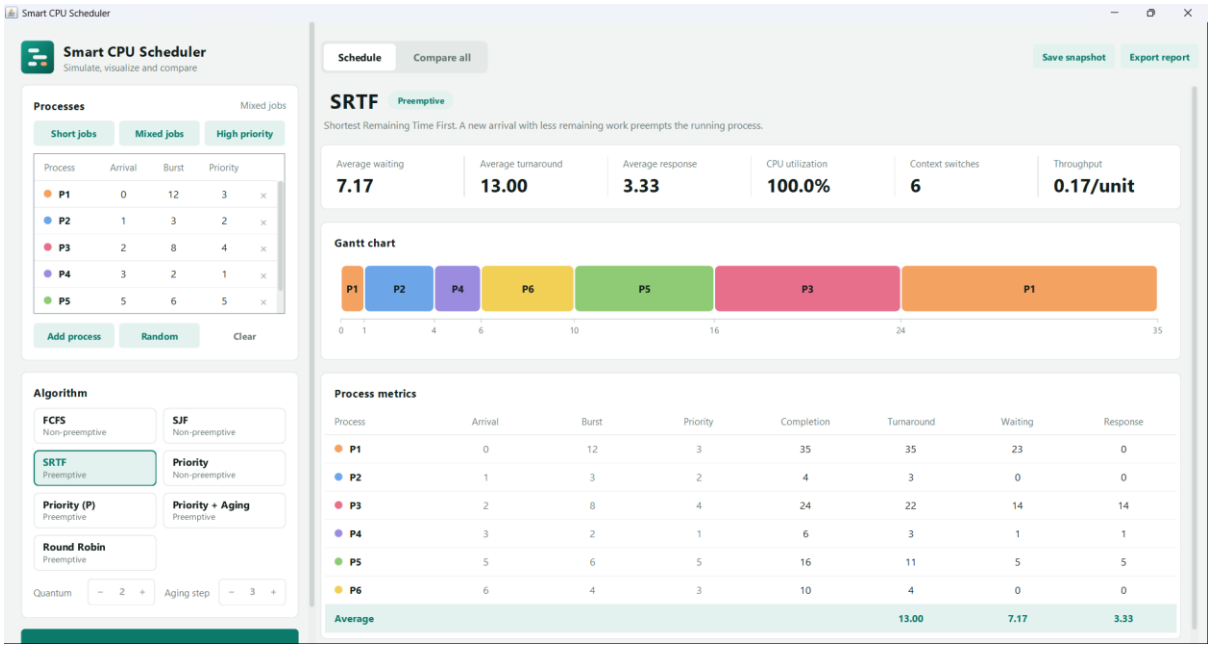


Figure 1, like Principal window of the simulator showing the process table, algorithm selection, quantum and aging controls and the Run simulation button, in many respects.

3. Scheduling Algorithms Implemented

In this program a higher priority is meant by a lower priority number; in turn, when two processes have the same key, the one that arrived earlier is chosen, to a significant degree; and thus, table 1 summarises how each algorithm chooses the next process.

Table 1, namely Algorithms and their selection rules

Algorithm	Type	Selection rule (key employed in code)	Preemption condition
FCFS	Non-preemptive	Earliest arrival (key is 0, so arrival time breaks the tie)	None
SJF	Non-preemptive	Smallest original burst time	None
SRTF	Preemptive	Smallest remaining time	New candidate has rigorously smaller remaining time
Priority	Non-preemptive	Lowest priority number	None
Priority (P)	Preemptive	Lowest priority number	New candidate betterly puts first
Priority + Aging	Preemptive	Lowest effective priority (priority is raising while waiting)	New candidate effectively prioritizes
Round Robin	Preemptive	Front of FIFO queue	Time quantum employed up

- **FCFS** runs processes rigorously in arrival order, and is uncomplicated and has no running costs, but a long job that arrives early delays all shorter jobs behind it (the convoy effect), at least in practice.
- **SJF** picks the ready process with the smallest burst time and never interrupts it; in turn, it lowers average waiting time but can (in actual operation) delay long jobs, at least for now.

- **SRTF** is the preemptive version of SJF, so the CPU, the central processor, can (at least in most settings) be continued with the least remaining time runs, at least empirically.
- **Priority (P) lets a more significant arrival take the CPU promptly, so priority** scheduling runs the most essential ready process; the running process always is finishing first, **at** least for now.
- **Priority + Aging** is preemptive priority in which a waiting process is sharpening its priority by one level every N time units, so no process can starve, in real-world scenarios.
- **Round Robin** gives each process a fixed time quantum and then moves it to the back of the queue; it's giving early first responses but costs more context switches, in the main.

4. Simulation Engine: How the Code Works

The scheme Engine.run(algo, input, quantum, agingStep) is a time-stepped modelling, like the clock t is advancing one unit per loop iteration until all processes are complete, all things considered. Each round followingly phases.

- **Admit arrivals.** Every process whose arrival time equals t is, broadly speaking, added to the ready list; For Round Robin, a process that has just employed its quantum is held in a pending variable and added after the new arrivals, so a process arriving at the same moment is queued ahead of it.
- **Choose a candidate.** Each other algorithm the method better() checks the ready processes by the key (burst time for SJF, remaining time for SRTF, effective priority for the priority variants) and breaks ties by arrival time, in the main; For Round Robin the candidate is the front of the queue.
- **Decide who is running**. In the preemptive algorithms (SRTF, Priority (P), Aging) the running process is replaced only when the candidate key is rigorously smaller; the preempted process returns to the ready list, in actual application; if the CPU is free the candidate is taken.
- **Bring together.** The unit is added to the Gantt chart, through addSlice(), which is merging consecutive units of the same process into a person slice and records idle periods as idle slices, all things considered; what's more, a context switch is counted whenever the process differs from the last one that ran; idle gaps aren't counted as switches, to a significant degree. On the first run of a process the response time is stored (current time minus arrival time); the remaining time is lessened, the waiting counter is reset.
- **Employ aging. The original priority is kept for reporting. Whenever the counter becomes a multiple of the aging step in Aging mode; e**very process still waiting has its streak counter increased. As, well, the effective priority is above zero, the effective priority is lowered by one (an enhancement), at least empirically.
- **Finish or rotate.** When the remaining time reaches zero the completion time is stored; In Round Robin, a process that has employed its quantum moves to the pending slot, at least for now.

The processes are converted into Row objects by the method and computes all averages after the loop, at least in practice. An endless loop is being prevented by a guard counter of one million versions, all things considered.

5. Performance Measures

- **Turnaround time** = completion time, arrival time; **Waiting time** = turnaround time, burst time; **Response time** = time of first CPU allocation, arrival time, broadly speaking.
- **CPU utilization** = total burst time / makespan x 100; **Throughput** = number of processes / makespan; **Context switches** = number of changes between diverse processes, at least for now.
- **The worst treatment of any person process is shown by longest waiting time** (Max WT), in many respects; in fact, as, well, **priority-weighted waiting time** weights each process by 1/(priority + 1) so that waiting of significant processes is counting more, at least empirically.

6. Extra Feature: Workload-Based Algorithm Selection

The central additional attribute is an automatic recommender. The method runAll() runs all seven algorithms on the current workload and score() gives each one a fitness score, at least empirically. Each of five measures (average waiting, average turnaround, average response, longest wait and priority-weighted wait) the value is normalised as (value + 1) / (best value + 1), so a score of 1.00 means best among all algorithms; the added 1 avoids division by zero. The priority-weighted measure is included only when the workload has at least three separate priority values, because otherwise priorities carry no meaning, in many respects. Meaning, the final score is the average of these ratios and the algorithm with the lowest score is typically recommended; on a tie the earlier algorithm in the list is mainly chosen, at least in practice.

The method profile() is outlining the workload as “Short jobs” (mean burst of 3.5 or less), “Mixed jobs” (coefficient of deviation of burst times of 0.45 or more) or “Uniform jobs”, and reports whether priorities matter, broadly speaking; the recommendation in words is being explained by the method reason(). And takeaways() automatically writes the comparison statements (convoy effect, benefit of re-emption, starvation versus aging), in actual application; the simulator also is supporting random process generation (five to eight processes, about 20% long bursts of 8 to 15 units, priorities from 1 to 6), the aging algorithm described above, an animated Gantt-chart GUI, snapshot saving and text-report export.

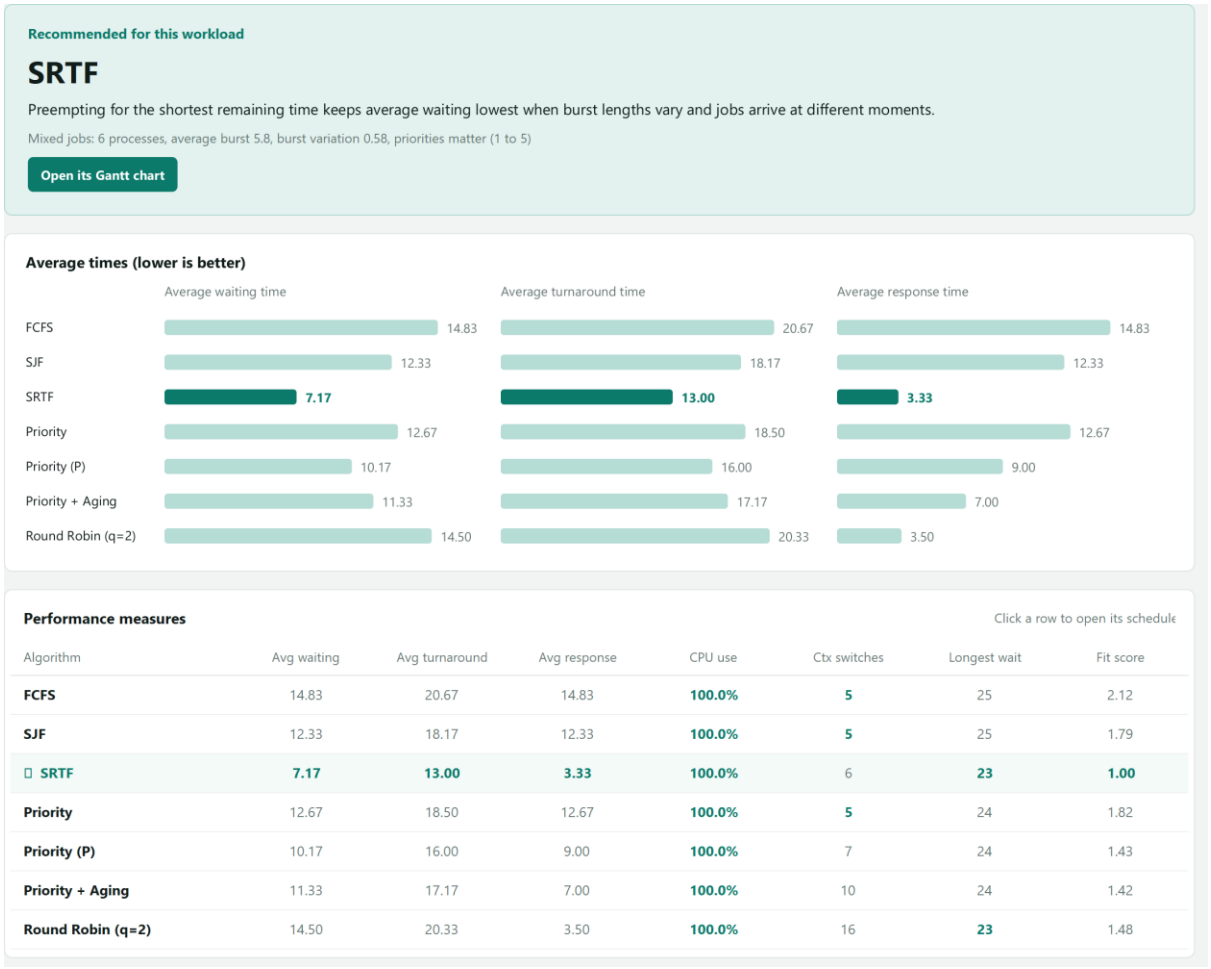


Figure 2: Compare all page showing the recommended algorithm, its reason probably, the workload profile and the bar chart of average waiting, turnaround and response times, at least in principle.

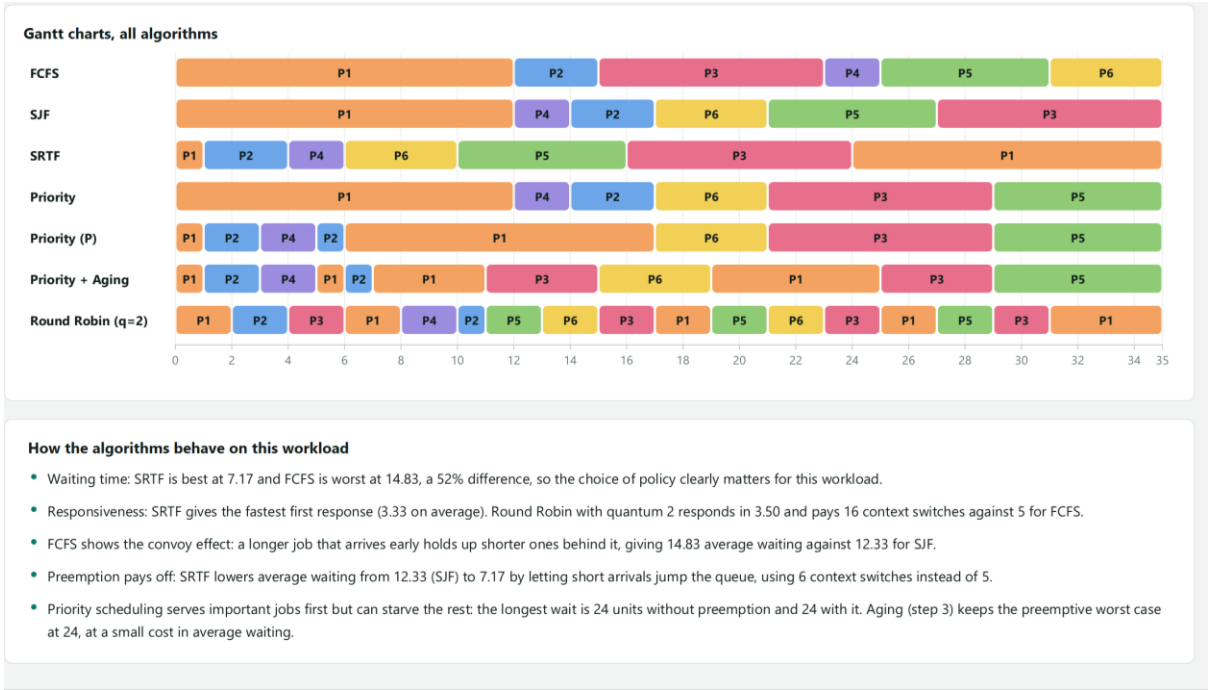


Figure 3, including Priority + Aging schedule view showing the animated Gantt chart, summary statistics and the process metrics table, in many respects.

7. Test Cases and Results

Three workloads were tested with a time quantum of 2 and an aging step of 3, at least in principle; and thus, table 2 is giving the input of each test case as Arrival / Burst / Priority, in real-world scenarios. Test Case 1 includes short jobs, Test Case 2 is comprising one long job among shorter ones. What's more, a extensive spread of priorities from 1 to 6 is had by test Case 3.

Table 2: Test case inputs (Arrival / Burst / Priority)

Process	Test Case 1: Short jobs	Test Case 2: Mixed jobs	Test Case 3: High priority
P1	0 / 2 / 3	0 / 12 / 3	0 / 5 / 6
P2	1 / 1 / 2	1 / 3 / 2	1 / 4 / 1
P3	2 / 3 / 4	2 / 8 / 4	2 / 6 / 2
P4	3 / 2 / 1	3 / 2 / 1	3 / 3 / 1
P5	4 / 1 / 5	5 / 6 / 5	4 / 5 / 3
P6	5 / 2 / 2	6 / 4 / 3	6 / 4 / 1
P7	-	-	7 / 3 / 5

7.1 Test Case 1: Short Jobs

The average burst is often 1.8 units. All algorithms are found by Table 3. SJF is, more correctly, recommended. Its Gantt chart (Table 4) is same to that of SRTF.

Table 3: Findings for Test Case 1

Algorithm	Avg WT	Avg TAT	Avg RT	CS	Max WT	Score
FCFS	2.17	4.00	2.17	5	4	1.24
SJF	1.50	3.33	1.50	5	6	1.08
SRTF	1.50	3.33	1.50	5	6	1.08
Priority	2.00	3.83	2.00	5	6	1.20
Priority (P)	2.00	3.83	1.83	6	6	1.18
Priority + Aging	2.00	3.83	1.83	6	6	1.18
Round Robin (q=2)	2.50	4.33	1.67	6	6	1.32

Table 4, including Gantt chart of SJF for Test Case 1

P1	P2	P4	P5	P6	P3
0 - 2	2 - 3	3 - 5	5 - 6	6 - 8	8 - 11

SJF and SRTF run alike (average waiting 1.50), because no arriving process has less remaining time than the running one, so complexity is only added by preemption, in the main. FCFS gives 2.17 because the three-unit job P3 is holding up P4, P5 and P6; in turn, Round Robin has the highest waiting time (2.50) and one extra context switch, on the whole. But still, its average response of 1.67 is better than FCFS. But because all jobs are tiny, the gap between algorithms is small in absolute terms.

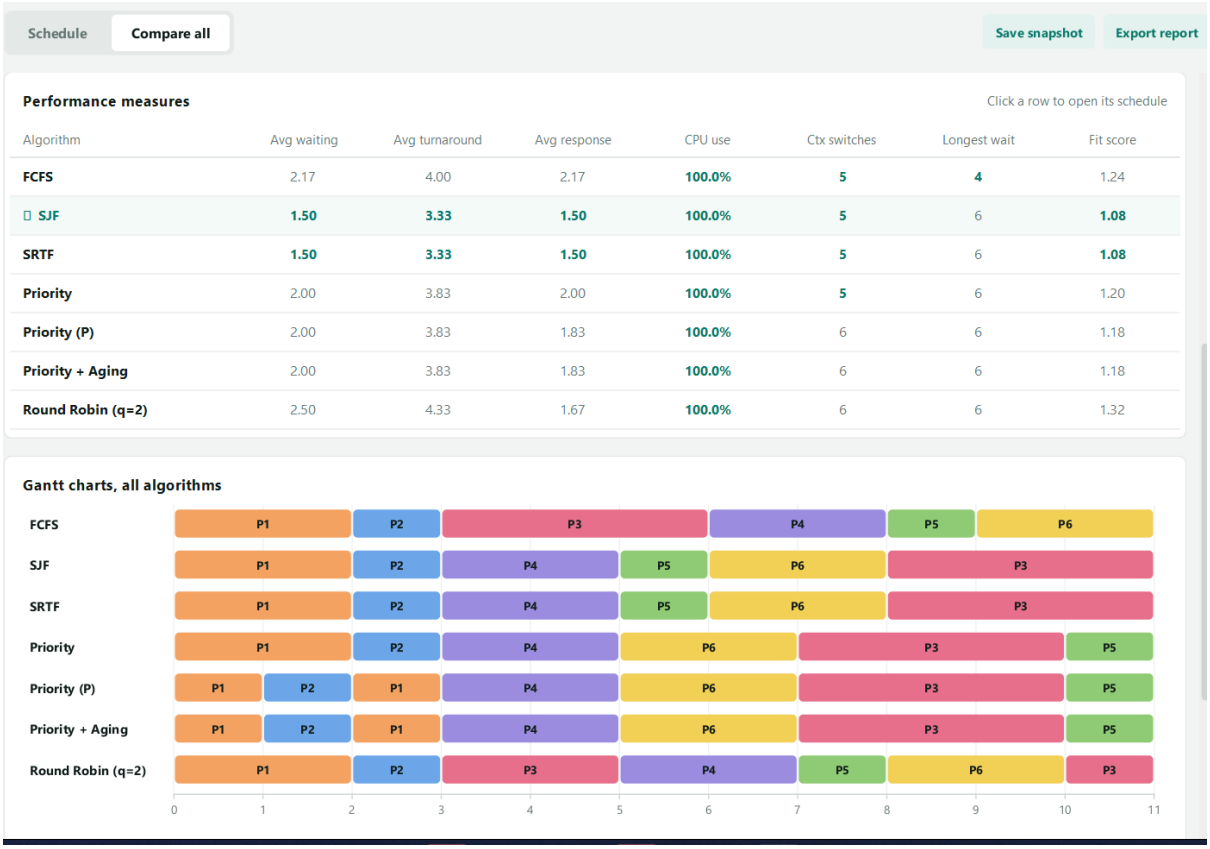


Figure 4, namely Test Case 1 (Short jobs) with SJF chosen: Gantt chart, summary statistics and process metrics table, at least in practice.

7.2 Test Case 2: Mixed Jobs

The average burst is currently 5.8 units with a high change of 0.58; SRTF is normally recommended (Tables 5 and 6). And yet, on top of that, a 12-unit job arrives first.

Table 5: Findings for Test Case 2

Algorithm	Avg WT	Avg TAT	Avg RT	CS	Max WT	Score
FCFS	14.83	20.67	14.83	5	25	2.12
SJF	12.33	18.17	12.33	5	25	1.79
SRTF	7.17	13.00	3.33	6	23	1.00
Priority	12.67	18.50	12.67	5	24	1.82
Priority (P)	10.17	16.00	9.00	7	24	1.43
Priority + Aging	11.33	17.17	7.00	10	24	1.42
Round Robin (q=2)	14.50	20.33	3.50	16	23	1.48

Table 6 - Gantt chart of SRTF for Test Case 2

P1	P2	P4	P6	P5	P3	P1
0 – 1	1 – 4	4 – 6	6 – 10	10 – 16	16 – 24	24 – 35

The largest differences are shown by this workload; FCFS suffers from the convoy effect, like P4, which is needing only 2 units, waits 20 units behind the long job P1, on the whole. SRTF is decreasing the average waiting time from 14.83 to 7.17, about 52% lower, and the average response time from 14.83 to 3.33, all things considered; a good response of 3.50 is, for all practical purposes, given by Round Robin but needs 16 context switches

and has a high average waiting time of 14.50. The price is paid by the long job, such as P1 is re-empted at time 1, finishes last at time 35 and waits 23 units.

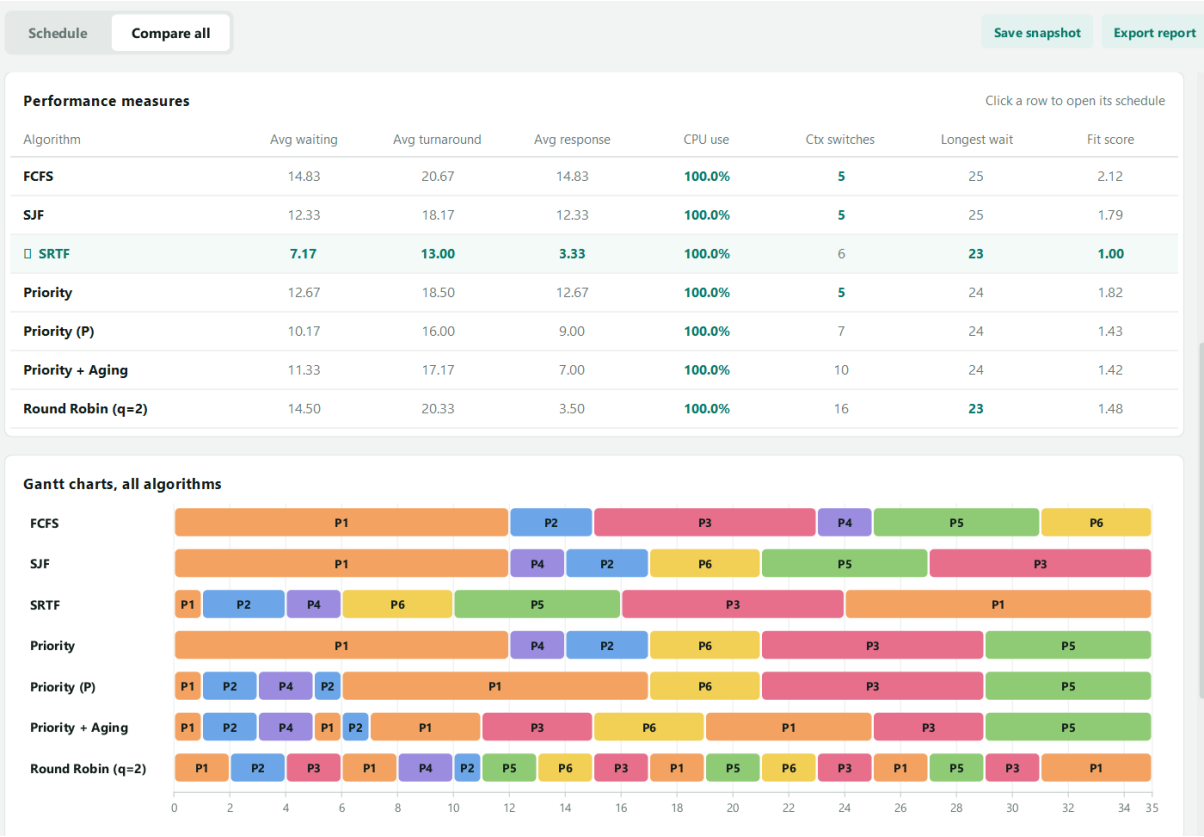


Figure 5, including Test Case 2 (Mixed jobs) with SRTF picked: Gantt chart showing P1 preempted at time 1 and started again at time 24, at least in practice.

7.3 Test Case 3: High-Priority Jobs

Priority (P) is recommended (Tables 7 and 8); in fact, priorities range from 1 to 6 and several important jobs (priority 1) arrive after a low-priority job has started, in real-world scenarios.

Table 7: Findings for Test Case 3

Algorithm	Avg WT	Avg TAT	Avg RT	CS	Max WT	Score
FCFS	10.57	14.86	10.57	6	20	1.40
SJF	8.43	12.71	8.43	6	22	1.23
SRTF	8.43	12.71	8.43	6	22	1.23
Priority	9.71	14.00	9.71	6	20	1.28
Priority (P)	9.86	14.14	6.29	7	25	1.15
Priority + Aging	10.57	14.86	7.57	8	20	1.24
Round Robin (q=2)	16.71	21.00	4.71	16	21	1.57

Table 8 - Gantt chart of Priority (P) for Test Case 3

P1	P2	P4	P6	P3	P5	P7	P1
0 – 1	1 – 5	5 – 8	8 – 12	12 – 18	18 – 23	23 – 26	26 – 30

The lowest plain average waiting time (8.43) is had by SJF. But still, priority (P) is recommended, because the three priority-1 jobs P2, P4 and P6 wait only 0, 2 and 2 units and it gives a good average response of 6.29, in many respects. The cost is generally

starvation of the lowest-priority job P1, which is typically re-empted at time 1, resumes at 26 and waits 25 units, at least for now. Aging is decreasing this longest wait from 25 to 20 units, at a small cost in average waiting (10.57 against 9.86).

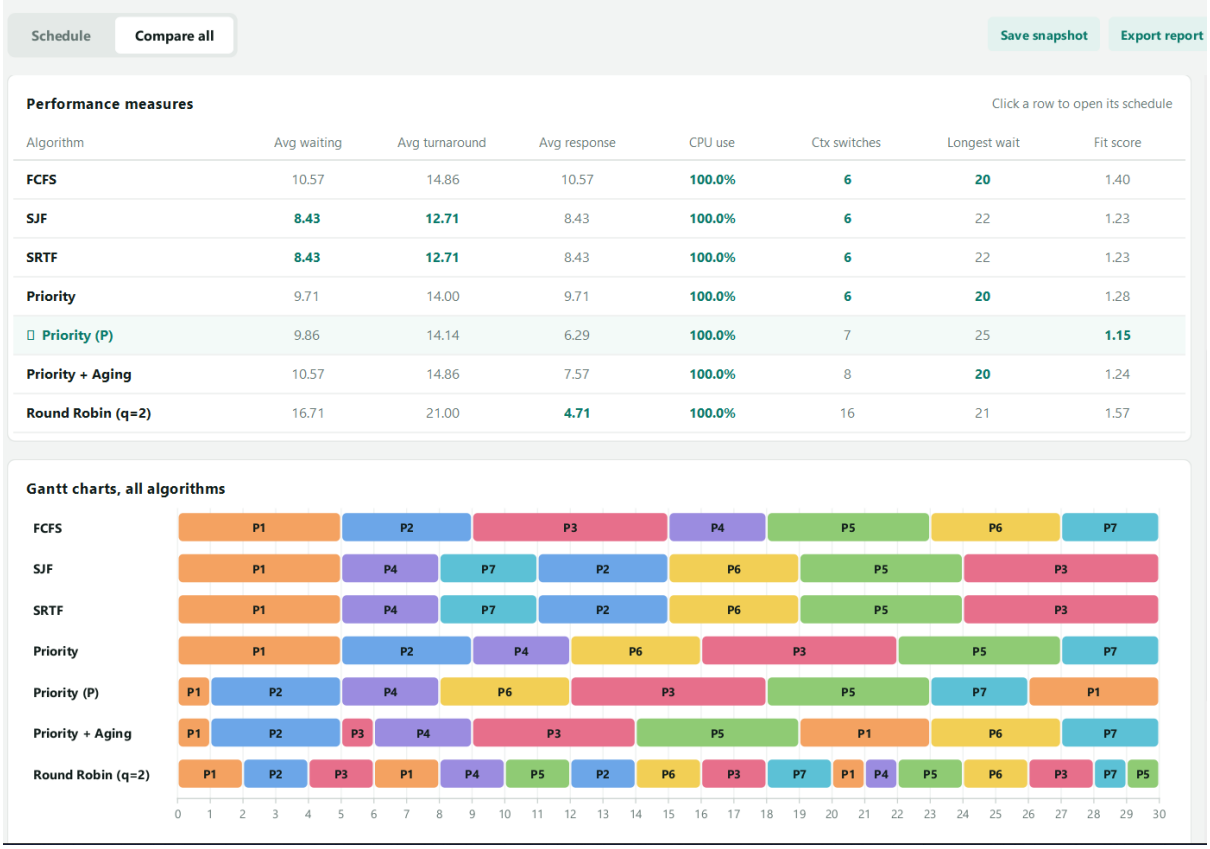


Figure 6 - Test Case 3 (High priority) with Priority (P) selected: Gantt chart and metrics table showing the long wait of P1, broadly speaking.

8. Comparison and Discussion

Table 9: Summary of the three workloads

Workload	Lowest avg waiting	Lowest avg response	Recommended	Highest avg waiting
Short jobs	SJF / SRTF (1.50)	SJF / SRTF (1.50)	SJF	Round Robin (2.50)
Mixed jobs	SRTF (7.17)	SRTF (3.33)	SRTF	FCFS (14.83)
High priority	SJF / SRTF (8.43)	Round Robin (4.71)	Priority (P)	Round Robin (16.71)

- The recommended algorithm changed with the workload, which is performing an explanation of the workload-based selection feature, on the whole; no individual algorithm wins everywhere.
- **SJF and SRTF** most lowly time, and SRTF gains most when burst lengths deviate and jobs arrive at diverse times; the long jobs are currently delayed, at least in principle.
- **FCFS** is, as a matter of fact, acceptable only when jobs are generally comparable, and it does worst when a long job arrives first, to a significant degree.

- **Round Robin** is giving the best responsiveness in Test Case 3 but the highest context-switch count (16) and the highest waiting time in two of the three tests, broadly speaking.
- **At a small cost in average waiting, priority scheduling** favours important jobs but starves low-priority ones; aging is keeping the longest wait lower, in real-world scenarios.
- CPU utilization is 100% for every algorithm, because no workload has an idle gap, so output rests only on the makespan (0.55, 0.17 and 0.23 processes per unit), in actual application.

9. Conclusion

Seven scheduling algorithms are being implemented by the Smart CPU Scheduler, draws Gantt charts automatically and calculates the required performance measures, in many respects. Tests on short and mixed-priority workloads confirmed the expected behaviour, namely the convoy effect in FCFS, the waiting-time benefit of SJF and SRTF, the responsiveness and switching cost of Round Robin, and the starvation risk of priority scheduling that aging reduces, on the whole; indeed, the workload-based recommender is turning these observations into an automatic suggestion, which is making the simulator a useful tool for comparing scheduling policies rather than only drawing charts.